FACULTAD DE INGENIERÍA

NATIONAL UNIVERSITY OF LA PLATA · FACULTY OF ENGINEERING

Chair (E1307) Introduction to Quantum Computer Architecture

Solving the N-Queens Problem using Grover's Algorithm

Ignacio Andres Schwindt (03229/0) · Friday, June 17, 2026

ABSTRACT

An implementation of Grover's quantum search algorithm applied to the combinatorial N-Queens problem is presented. The encoding assigns $k = \lceil \log_2 N \rceil$ qubits per queen to represent its column, eliminating row conflicts by construction. The oracle detects column and diagonal conflicts through explicit pattern enumeration and flag qubits, applying a -1 phase to valid configurations via the *compute–phase–uncompute* scheme. Full correctness is verified for $N = 4$ (14 qubits, 8 iterations, $P(\text{success}) \approx 99.3\%$) and the oracle is validated in isolation for $N = 5$ (30 qubits), confirming that all 10 known solutions are correctly identified. Scalability limitations and possible optimizations using quantum adders are discussed.

KEYWORDS *Quantum computing · Grover's algorithm · N-Queens · Quantum oracle · Qiskit · Unstructured search*

1. INTRODUCTION

The N-Queens problem consists of placing N queens on an $N \times N$ chessboard such that no two queens threaten each other: they cannot share the same row, column, or diagonal. Originally formulated by Max Bezzel in 1848 and extensively studied by Gauss, the problem has become a classic in combinatorial theory and computer science [**?**]. For $N = 4$ there are exactly 2 solutions; for $N = 5$, there are 10; for $N = 8$ (standard chessboard), 92; and the number grows rapidly. Classical brute-force search has an $\mathcal{O}(N!)$ complexity when exploring column permutations, and although backtracking algorithms improve this in practice, the worst-case complexity remains exponential.

Grover's algorithm [**?**], published in 1996, offers a quadratic speedup for unstructured search problems: if there are M solutions in a search space of size $\mathcal{N}$, it finds them in $\mathcal{O}(\sqrt{\mathcal{N}/M})$ oracle queries, compared to $\mathcal{O}(\mathcal{N}/M)$ in the classical case. This speedup is optimal: Bennett, Bernstein, Brassard, and Vazirani proved that no quantum algorithm can do better for unstructured search [**?**].

In this work, a Python implementation using Qiskit [**?**] and simulated with Aer is presented, covering the quantum encoding of the board, oracle design, Grover's diffuser, and results for $N = 4$ (full execution) and $N = 5$ (oracle validation).

Additionally, the impact of mapping combinatorial constraints to reversible gates is discussed. Although the physical execution of algorithms with such depth is still limited by the noise inherent to NISQ (*Noisy Intermediate-Scale Quantum*) processors, this exploration establishes a useful case study on how to modularly structure and validate complex decision oracles.

2. GROVER'S ALGORITHM

2.1 Principle and General Flow

Grover operates on n qubits with $\mathcal{N} = 2^n$ states. The oracle U_f applies $U_f |x\rangle = (-1)^{f(x)} |x\rangle$ (*phase kickback* to the solutions). The algorithm (Fig. **??**) consists of three stages: **(1)** Initialization in uniform superposition $|\psi_0\rangle = H^{\otimes n} |0\rangle^{\otimes n}$. **(2)** Grover iteration (repeated t^* times): oracle U_f + diffuser $D = 2 |\psi_0\rangle\langle\psi_0| - I$. **(3)** Measurement in the computational basis.

Measurement collapses the system state, revealing the solution with a high probability if the correct number of iterations was chosen. This probabilistic behavior is characteristic of quantum algorithms, where constructive interference amplifies the amplitude of states that satisfy the oracle conditions.

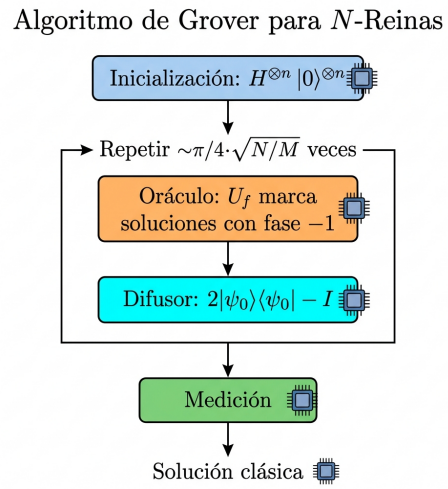

Figure 1. Flow of Grover's algorithm for N-Queens.

2.2 Geometric Interpretation

The system state lives in a two-dimensional plane generated by:

$$|\alpha\rangle = \frac{1}{\sqrt{\mathcal{N} - M}} \sum_{f(x)=0} |x\rangle, \quad |\beta\rangle = \frac{1}{\sqrt{M}} \sum_{f(x)=1} |x\rangle \tag{1}$$

The initial state $|\psi_0\rangle$ forms an angle $\theta/2 = \arcsin\sqrt{M/\mathcal{N}}$ with $|\alpha\rangle$. Each iteration rotates the state by an angle θ towards $|\beta\rangle$.

2.3 Optimal Iterations

The optimal number of iterations is:

$$t^* = \left\lfloor \frac{\pi}{2\theta} - \frac{1}{2} \right\rfloor, \quad \theta = 2 \arcsin\sqrt{\frac{M}{\mathcal{N}}} \tag{2}$$

For $N = 4$: $\mathcal{N} = 256$, $M = 2$, $\theta \approx 0.177$ rad, $t^* = 8$, $P(\text{success}) \approx 99.3\%$. **Note:** If t^* is exceeded, the probability *decreases* — the algorithm "overshoots" the solution. This phenomenon has no classical analog.

3. QUANTUM ENCODING

3.1 Row Symmetry Elimination

The first design decision is to implicitly fix each queen i in row i. This eliminates row conflicts by construction and reduces the search space from N^{2N} (arbitrary positions) to N^N (only columns per queen).

3.2 State Register

The column of queen i is encoded in $k = \lceil \log_2 N \rceil$ binary qubits. The qubits $q_{ik}, q_{ik+1}, \ldots, q_{ik+k-1}$ represent the column in base 2, with q_{ik} as the least significant bit (LSB). For $N = 4$: $k = 2$, total register of $n = Nk = 8$ qubits (Fig. **??**).

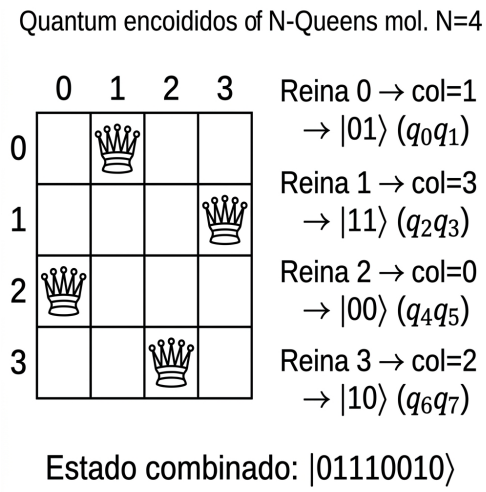

Figure 2. Encoding of $[1,3,0,2]$ for $N=4$: 2 qubits per queen.

3.3 Ancilla Qubits (Flags)

The oracle requires auxiliary qubits: **Pair flags**: one qubit for each pair (i,j) with $i<j$, totaling $\binom{N}{2}$ flags. **Range flags** (only when $2^k > N$): one qubit per queen to detect columns outside $[0,N)$. For $N=4$, $2^2=4=N$, they are not needed; for $N=5$ ($2^3=8>5$), 5 flags are added.

Table 1. Circuit resources according to N.

N	k	State	P. flags	R. flags	Total	Sols.
4	2	8	6	0	14	2
5	3	15	10	5	30	10
6	3	18	15	6	39	4

4. ORACLE DESIGN

The oracle is the core component of the circuit. It applies a -1 phase exclusively to states representing valid configurations, in three stages (Fig. **??**): *compute*, *phase flip*, and *uncompute*.

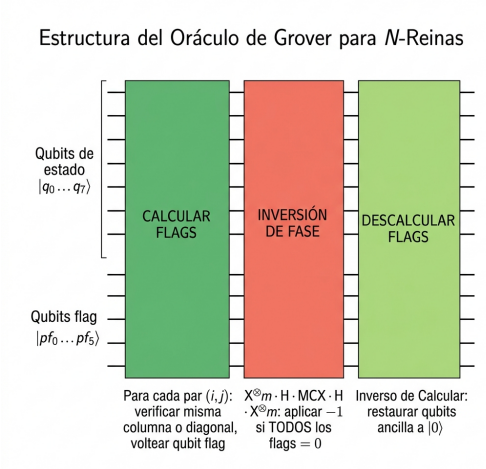

Figure 3. Structure: Compute → Phase Flip → Uncompute.

4.1 Compute Flags

For each pair (i,j), $i<j$, a conflict is verified ($c_i = c_j$ or $|c_i - c_j| = j-i$) using **explicit pattern enumeration** (Fig. **??**).

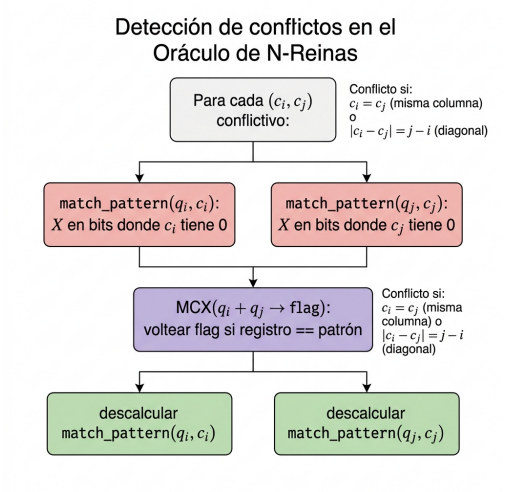

Figure 4. Conflict detection (c_i, c_j) with `match_pattern`.

4.1.1 *Function match_pattern*

This is the key piece for detection. Given a k-qubit register and a target value v, it applies X gates to each qubit whose corresponding bit in v is 0:

$$\bigotimes_{b=0}^{k-1} X^{1-v_b} |v\rangle = |1\rangle^{\otimes k} \tag{3}$$

This transforms the condition "the register equals v" into "all qubits equal 1", which is exactly what an MCX (generalized Toffoli) gate detects. A subsequent MCX flips the corresponding flag. Finally, the same sequence of X gates is applied to restore the register ($X^2 = I$). For a fixed computational state $\left|c_i, c_j\right\rangle$, exactly one pattern combination matches, ensuring the flag is flipped 0 or 1 times.

4.1.2 *Out-of-Range Detection*

For an N that is not a power of 2 (e.g., $N=5$ with $k=3$), the column values $\{N, N+1, \ldots, 2^k-1\}$ are invalid. An additional flag per queen is activated if its column encodes a value $\geq N$, using the same `match_pattern` technique.

4.2 Phase Flip

Once all flags are computed, a -1 phase is applied conditional on all of them being 0 (no conflicts and no out-of-range columns):

$$X^{\otimes m} \cdot \left(H \cdot \text{MCX} \cdot H\right) \cdot X^{\otimes m} \tag{4}$$

where m is the total number of flags. The sequence $X^{\otimes m}$ converts $|0\rangle^{\otimes m}$ to $|1\rangle^{\otimes m}$ (which triggers the MCX). The trick $H \cdot \text{MCX} \cdot H$ implements an MCZ (controlled phase):

$$HXH = Z \implies H \cdot \text{MCX}(\vec{c} \to t) \cdot H = \text{MCZ}(\vec{c}, t) \tag{5}$$

4.3 Uncompute Flags

For the oracle to be unitary and reusable across iterations, the ancillas must return to $|0\rangle$. The Compute stage is undone by applying the same operations in reverse order. Since X and MCX are self-inverse, this inversion is straightforward.

5. GROVER'S DIFFUSER

It implements the reflection $D = 2|\psi_0\rangle\langle\psi_0| - I$ (inversion about the mean) on the n state qubits. Its gate decomposition is:

$$D = H^{\otimes n} \cdot X^{\otimes n} \cdot \text{MCZ} \cdot X^{\otimes n} \cdot H^{\otimes n} \tag{6}$$

The circuit (Fig. **??**) operates by: (1) moving to the Hadamard basis with $H^{\otimes n}$; (2) marking $|0\ldots0\rangle$ with $X^{\otimes n}$ followed by MCZ; (3) undoing $X^{\otimes n}$ and returning to the computational basis.

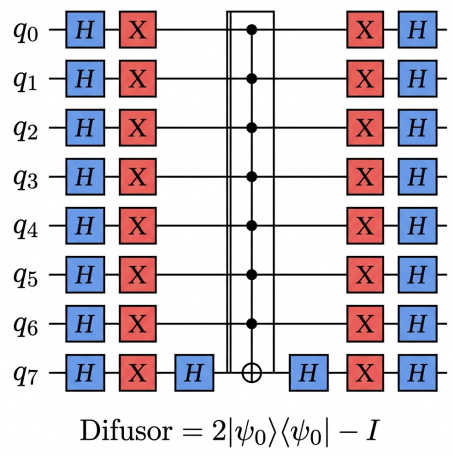

Figure 5. Diffuser circuit for 8 state qubits.

Effect on amplitudes. Let $\bar{a}$ be the mean amplitude. The diffuser transforms each amplitude $a_x \to 2\bar{a} - a_x$: states with amplitudes below the mean are attenuated, while solutions (with large negative amplitudes) are reflected to large positive values.

6. RESULTS

6.1 $N = 4$: Full Execution

For $N = 4$, the complete circuit was simulated with the `AerSimulator` backend (`statevector` method) for 4096 shots.

Table 2. Circuit parameters for $N = 4$.

Parameter	Value
State qubits	8 (2 bits/column)
Pair flags	6
Total qubits	14
Search space	$2^8 = 256$
Classical solutions	2
Optimal iterations	8
Depth (transpiled)	∼41 000

The two valid solutions, $[1,3,0,2]$ and $[2,0,3,1]$, account for $\sim$99.3% of the total probability ($\sim$49.6% each, Fig. **??**). Decoding the measured bitstring is done by reversing the bit order (Qiskit prints MSB first) and reconstructing the column of each queen: $\text{col}_i = \sum_{b=0}^{k-1} c_{ik+b} \cdot 2^b$.

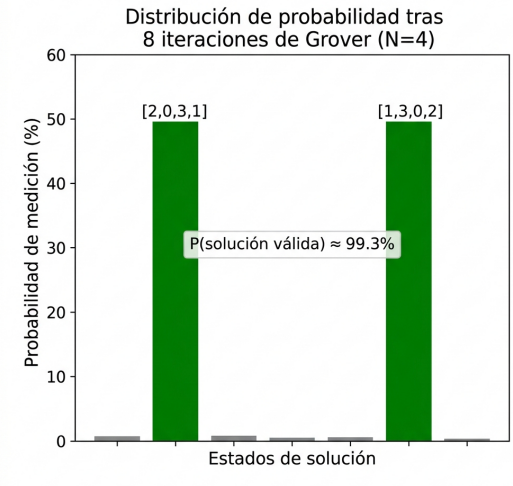

Figure 6. Probability after 8 iterations ($N = 4$). The two solutions dominate with ∼49.6% each; noise $< 1\%$.

6.2 $N = 5$: Oracle Validation

The full circuit for $N = 5$ requires 30 qubits and ∼44 iterations ($\sim 6 \times 10^5$ gates), which is beyond the practical reach of classical simulation. However, the oracle was validated in isolation by preparing known computational states and measuring the flags:

The **10 classical solutions** (obtained by brute force using `classical_solutions`) resulted in 0 active flags in all cases. Tested negative cases included:

- $[0,0,0,0,0]$: all in the same column.
- $[0,1,2,3,4]$: main diagonal.
- $[0,2,4,1,7]$: column 7 out of range (≥ 5).
- $[1,3,5,0,2]$: column 5 out of range.

All were correctly rejected with flags > 0. This confirms that the oracle perfectly separates valid configurations from the rest of the search space.

7. COMPLEXITY AND LIMITATIONS

7.1 Oracle Complexity

Explicit pattern enumeration produces: $\binom{N}{2} = \mathcal{O}(N^2)$ pairs of queens; up to $\mathcal{O}(N)$ conflicting combinations per pair; each verification: $\mathcal{O}(k)$ X gates plus an MCX with $2k$ controls. Total: $\mathcal{O}(N^3)$ MCX calls and $\mathcal{O}(N^3 \log N)$ elementary gates. An implementation using **quantum adders** would allow columns to be compared arithmetically, reducing the MCX gates per pair from $\mathcal{O}(N)$ to $\mathcal{O}(\text{poly}(k))$.

7.2 Scalability

Table 3. Simulation feasibility according to N.

N	Qubits	Iters.	Simulable
4	14	8	Yes (`statevector`)
5	30	44	Oracle only (`MPS`)
6	39	∼600	No (classical simulator)

For $N \geq 5$, the two paths towards end-to-end execution are: (1) real quantum hardware with sufficient fidelity and T_1/T_2 times (aspirational in 2026); or (2) rewriting the oracle with quantum adders to reduce circuit depth.

8. CONCLUSIONS

1. **Verified Correctness.** For $N = 4$, the complete algorithm finds the 2 solutions with $P \approx 99.3\%$. For $N = 5$, the oracle correctly classifies the 10 solutions and rejects all tested invalid cases.
2. **Quadratic Speedup.** The algorithm queries the oracle $\mathcal{O}(\sqrt{\mathcal{N}/M})$ times, compared to $\mathcal{O}(\mathcal{N}/M)$ in classical search. For $N = 4$: 8 iterations vs. ∼128 classical queries.
3. **Modular Design.** The separation into encoding, oracle (compute–phase–uncompute), and diffuser allows extending or replacing components independently.
4. **Main Limitation.** Explicit enumeration generates deep circuits (∼41 000 depth for $N = 4$). Quantum adders would significantly reduce depth for $N \geq 5$.
5. **Perspective.** Execution on real quantum hardware would require coherence times (T_1, T_2) far exceeding currently available ones, along with quantum error correction. Nonetheless, the implementation serves as a proof-of-concept for Grover's quadratic advantage on combinatorial problems.

REFERENCES

[1] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proc. 28th ACM STOC*, pp. 212–219, 1996.

[2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge Univ. Press, 10th Ann. Ed., 2010.

[3] Qiskit Development Team, "Qiskit: An open-source framework for quantum computing," https://qiskit.org, 2024.

[4] J. Bell and B. Stevens, "A survey of known results and research areas for n-queens," *Discrete Math.*, vol. 309, no. 1, pp. 1–31, 2009.

[5] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1510–1523, 1997.